

Beyond Accuracy: Counterfactual Auditing of Long-Term Agent Memory

Tyler Chan

chen7751@umn.edu

Abstract

Long-term agent memory helps LLM-based systems remember past user–assistant conversations and use this history in future interactions. This ability is becoming common in real systems, including coding assistants and personal agents. Many recent benchmarks evaluate these memory systems with downstream accuracy, but accuracy alone cannot explain why a memory system succeeds or fails. Two agents may have similar accuracy while using memory in very different ways: one may retrieve the right memory and ground the answer on it, while another may retrieve stale or conflicting evidence and still produce a correct-looking answer. In this project, we study this hidden behavior with counterfactual memory auditing. We first run each agent normally and trace answer-stage evidence back to the original memory writes. Then we remove one traced evidence write and replay the same question to measure how the final answer changes. Across six long-term memory agents on LongMemEval, we find that benchmark accuracy hides important memory failure modes: some high-accuracy systems are highly sensitive to single memory writes, correct retrieval does not always imply correct causal control, and some systems perform poorly because their memory design is not aligned with fine-grained factual QA. These results suggest that long-term memory systems should be evaluated not only by final accuracy, but also by how their memory writes causally control answers. The code can be found at <https://github.com/TylerChan/Counterfactual-Memory-Auditing-for-Long-Term-Agents>.

1 Introduction

LLM-based agents are increasingly expected to remember information across long interactions. Instead of answering each user message as an isolated request, these agents maintain memory over past user–assistant conversations and use it to support personalization, long-term task continuity, and

cross-session reasoning. This makes memory an important component in practical agent systems. If the memory system fails, the agent may forget important user information, rely on outdated facts, or produce answers that look plausible but are grounded in the wrong history.

Recent benchmarks such as LongMemEval and LoCoMo evaluate this ability by testing whether an agent can answer questions from long interaction histories (Wu et al., 2025; Maharana et al., 2024). This is useful because it gives a clear downstream accuracy score. However, accuracy alone cannot explain why a memory system succeeds or fails. It only tells us whether the final answer matches the gold answer. It does not tell us whether the right memory was retrieved, whether stale or conflicting memories affected the answer, or which memory item actually controlled the final response.

Some recent work has started to use counterfactual stress tests for memory agents. For example, THEANINE introduces TeaFarm, where the evaluator asks counterfactual questions with false premises and checks whether the agent can avoid being misled by them (Ong et al., 2025). This is useful because it tests whether the agent can reference past conversations when the current user input is misleading. However, it still does not answer a more fine-grained question: which stored memory write caused the final answer? Our work is motivated by this gap. Instead of only changing the query, we intervene on the memory writes themselves and measure how the answer changes.

This creates a problem for memory system evaluation. Two agents may have similar accuracy but very different internal memory behavior. One agent may retrieve the correct memory and directly ground the final answer on it. Another agent may retrieve the correct memory but still be influenced by another stale or conflicting memory. A third agent may not use memory effectively at all, but still answer some questions correctly because of

language-model priors or prompt-level guessing. These cases look similar under final accuracy, but they imply very different memory design problems.

In this project, we study this hidden behavior with counterfactual memory auditing. The key idea is simple: if a memory write really supports an answer, then removing that write should change the answer or make the agent less able to answer. We first run each agent normally on LongMemEval and trace answer-stage evidence back to the original memory writes that produced it. Then we remove one traced evidence write and replay the same question. By comparing the baseline answer and the counterfactual answer, we can measure how much the final response depends on specific memory writes.

We evaluate six long-term memory agents with this framework. Our results show that benchmark accuracy hides important memory failure modes. Some high-accuracy systems are highly sensitive to removing a single evidence write, which suggests that their answers are strongly routed through a small number of memory items. We also find that correct retrieval is not always the same as correct causal control: an agent may fetch the right memory, but that memory may not dominate the final answer. Finally, the results suggest that memory design should be aligned with the target application. Agents designed for dialogue continuity or persona simulation do not necessarily perform well on fine-grained factual memory QA, even when they have sophisticated memory structures.

2 Related Work

Recent work has started to build benchmarks for long-term memory agents. LoCoMo studies very long-term conversational memory across multi-session dialogues (Maharana et al., 2024). LongMemEval evaluates chat assistants on long-term interactive memory abilities such as information extraction, multi-session reasoning, temporal reasoning, knowledge update, and abstention (Wu et al., 2025). More recent benchmarks such as AMemGym and MemoryAgentBench move toward interactive and incremental memory evaluation (Cheng et al., 2026; Hu et al., 2026). These benchmarks are important because they make memory performance measurable. However, their main signal is still whether the final answer is correct. They do not directly explain which stored memory caused the answer.

Another line of work designs memory architectures for LLM agents. MemoryOS and Mem0 build general memory systems for storing, updating, and retrieving long-term user information (Kang et al., 2025; Chhikara et al., 2025). THEANINE uses timeline-based memory management to preserve temporal and causal relations between memories (Ong et al., 2025). SHARE focuses on shared memories between speakers in open-domain dialogue (Kim et al., 2025). LD-Agent studies long-term personalized dialogue with event and persona memory (Li et al., 2025). AnnaAgent uses multi-session memory to simulate realistic mental-health seekers (Wang et al., 2025). These systems show that memory is not a single design problem. Different applications need different memory signals, so downstream accuracy alone is not enough to explain whether a memory system is using the right information.

The closest related direction is counterfactual memory evaluation. THEANINE introduces TeaFarm, where the evaluator asks counterfactual questions with false premises and checks whether the agent can avoid being misled by the current input (Ong et al., 2025). This tests whether the agent can use past memory when the query itself is misleading. Our work asks a different question. Instead of changing the query, we change the memory state itself. We remove one traced evidence write and ask whether the final answer changes. This gives a more direct view of memory-write-level causal influence.

3 Counterfactual Memory Auditing

The goal of our method is to measure whether a specific memory write has causal influence on the final answer. The intuition is simple. If an answer really depends on one memory write, then removing that write should change the answer, change the retrieved evidence, or make the agent unable to answer. This is different from normal benchmark evaluation, which only checks whether the final answer is correct.

3.1 Evaluation Setup

We sample 100 balanced questions from the original LongMemEval dataset in our main run. Each question contains a long user–assistant history before the final question. In this subset, one question requires replaying about 49 prior sessions on average, with about 249 user–assistant message

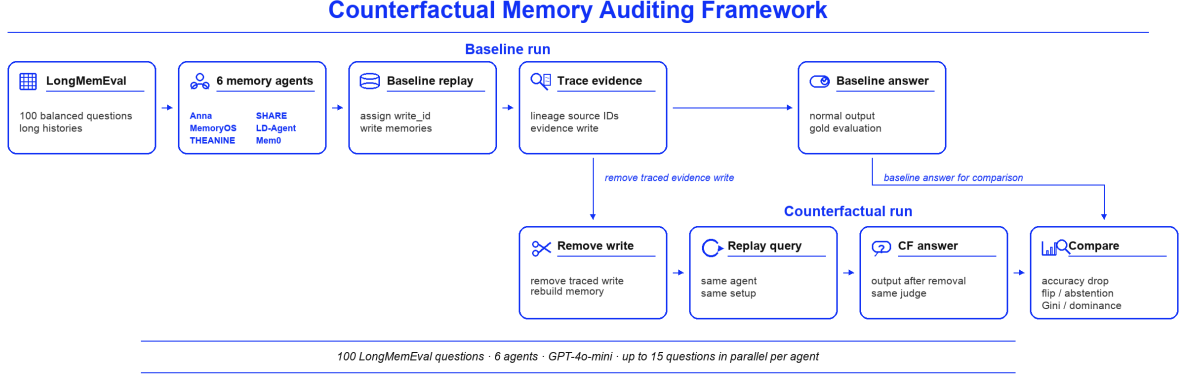


Figure 1: Overview of the counterfactual memory auditing framework. We first run each agent normally, trace answer-stage evidence back to original memory writes, remove one traced evidence write, replay the same question, and compare the baseline and counterfactual outputs.

pairs. We evaluate six memory agents: AnnaAgent, SHARE, MemoryOS, LD-Agent, THEANINE, and Mem0. For the LLM calls in the unified evaluation pipeline, we use GPT-4o-mini and the same answer prompt format across agents when possible; the full prompt is shown in Appendix 3. This makes the comparison focus more on the memory pipeline rather than on different answer prompts.

3.2 Tracing and Counterfactual Replay

For each question, we first run a normal baseline replay. During this step, the agent receives all prior user–assistant messages in chronological order and writes memories according to its own memory mechanism. Before a message enters the memory pipeline, we assign it a unique `write_id`. If an agent later summarizes, merges, links, or retrieves a memory, we keep the source information through `lineage_source_ids`. In this way, an evidence item shown to the answering stage can be traced back to the original memory writes that produced it.

After the baseline run, we build counterfactual runs from the traced answer-stage evidence. We focus on evidence writes that actually reach the prompt or retrieval context used for answering. For one counterfactual run, we remove one traced evidence write and replay the same question under the same agent. The removal happens before the memory pipeline writes or constructs downstream memories, so the counterfactual run tests how the whole memory pipeline changes when that source write is missing. We then compare the baseline answer with the counterfactual answer.

Agent	Questions	Avg. Min. per Audited Question
AnnaAgent	100	0.9
SHARE	100	190.1
MemoryOS	100	349.9
LD-Agent	100	312.3
THEANINE	100	590.9
Mem0	100	75.3

Table 1: Approximate MSI runtime per audited question, including baseline and counterfactual replay

3.3 Metrics and Runtime

Counterfactual accuracy measures whether removing traced evidence hurts final LongMemEval performance. **Answer flip rate** measures how often the answer changes after one evidence write is removed. **Abstention flip rate** measures whether the agent changes into or out of “I don’t know.” **Counterfactual Gini** measures whether influence is concentrated in a small number of writes. **ETDL** measures how far back in time the oldest dominant memory write is. Finally, the **retrieval–dominance matrix** asks whether the gold memory was retrieved and whether it actually dominated the final answer. This last metric is important because correct retrieval and correct causal control are not the same thing.

Table 1 shows the practical bottleneck. Without parallel shards, 100 questions would take about 5 days for Mem0, 22–24 days for LD-Agent and MemoryOS, and about 41 days for THEANINE. Running all six systems sequentially would take about three months, so we used up to 15 parallel shards per agent on MSI. This is why configurable benchmarks such as AMemGym and MemoryA-

gentBench are useful: they make memory evaluation cost easier to control (Cheng et al., 2026; Hu et al., 2026). We also include an API cost estimate in Appendix 7.

4 Results and Analysis

4.1 Counterfactual Accuracy Drops

Table 2 shows the first-level result. Removing traced evidence writes hurts most agents, but the size of the drop is not uniform. Anna has the largest drop, but its baseline setting only has 75 counterfactual queries with exposed evidence. Among the stronger memory systems, THEANINE, Mem0, and MemoryOS all lose accuracy after the counterfactual intervention. This fits their design: MemoryOS and Mem0 are built as general long-term memory systems, and THEANINE uses timeline-based memory (Kang et al., 2025; Chhikara et al., 2025; Ong et al., 2025). Because their answers are routed through explicit memory structures, removing traced evidence is more likely to hurt final accuracy.

The lower scores for SHARE and LD-Agent should not be read only as weak memory. Their original goals are open-domain shared dialogue memory and long-term personalized dialogue, not fine-grained factual QA (Kim et al., 2025; Li et al., 2025). LongMemEval is rigorous because it asks factual and temporal questions over long histories, so the memory system must preserve small facts, timestamps, and cross-session counts. Anna shows the same application-alignment issue from another angle. It has transferable multi-session memory, so its score is not extremely low. However, its original goal is realistic mental-health seeker simulation, where the important signals are emotional state, evolving complaints, and counseling continuity (Wang et al., 2025). This suggests that a good memory system should not only store more information. It should capture the memory signals that matter for the target application.

4.2 Sensitivity Failure Modes

The same accuracy drop can come from different failure modes. Table 3 separates these effects. MemoryOS has the highest answer flip rate, which means many of its final answers change when one evidence write is removed. THEANINE has the highest abstention flip rate, which means removing evidence often changes whether it answers or says “I don’t know.” These two metrics are the

Agent	Baseline	Counterfactual	
		Accuracy	Drop
Anna	0.39	0.17	-0.21
THEANINE	0.35	0.20	-0.15
Mem0	0.43	0.33	-0.10
MemoryOS	0.40	0.31	-0.09
SHARE	0.15	0.09	-0.06
LD-Agent	0.10	0.08	-0.02

Table 2: Paired LongMemEval accuracy under baseline and counterfactual evaluation. Counterfactual Accuracy removes the traced evidence write selected for each query. Drop is Counterfactual Accuracy minus baseline.

Agent	Answer Flip Rate	Abstention Flip Rate	Counterfactual Gini
Anna	0.16	0.10	0.25
THEANINE	0.43	0.27	0.11
Mem0	0.30	0.14	0.20
MemoryOS	0.50	0.21	0.21
SHARE	0.10	0.10	0.03
LD-Agent	0.03	0.03	0.05

Table 3: Counterfactual sensitivity summary. **Answer Flip Rate** measures how often removing one evidence write changes the answer; **Abstention Flip Rate** measures changes into or out of “I don’t know”; **Counterfactual Gini** measures whether counterfactual influence is concentrated in a few writes.

main signal in this table. Counterfactual Gini is less central here, but it adds a supporting view: it shows whether the effect comes from a few highly influential writes rather than many small changes.

4.3 Directional Answer Changes

Table 4 explains how the counterfactual accuracy drop happens. The most important direction is correct-to-wrong, because it means the removed evidence write was supporting a correct answer. MemoryOS has the largest number of correct-to-wrong cases, but it also has several wrong-to-correct cases.

This is consistent with the MemoryOS retrieval design. Its retriever searches mid-term pages and long-term user/assistant knowledge by similarity, keeps high-scoring items, and then places these items into the final answer prompt. This is a strong recall-oriented design, but it is not a conflict resolution layer. A retrieved item can be relevant but still stale, competing, or misleading. Therefore, removing one evidence write can break a correct answer, but it can also remove misleading evidence and make a wrong answer correct. The answer-to-IDK column is also useful. It shows when the

Agent	N	C→W	W→C	Ans.→IDK	IDK→Ans.
Anna	75	19	3	19	2
SHARE	100	7	1	9	2
MemoryOS	100	21	12	25	7
LD-Agent	100	3	1	3	2
THEANINE	75	18	7	22	5
Mem0	98	18	8	19	15

Table 4: Directional changes between baseline and counterfactual answers. C and W denote evaluator-labeled correct and wrong answers.

agent loses answerability after the evidence write is removed.

4.4 Temporal Horizon Alignment

ETDL measures the time gap between the query and the dominant evidence write found by the counterfactual intervention. We do not use raw ETDL as a standalone robustness score. The reason is that LongMemEval is a replay benchmark. For each question, the agent replays timestamped historical conversations, answers the question, and then we evaluate the next question separately. Therefore, a large ETDL does not mean that the deployed agent actually lived with a user for several months. It only means that, inside this replayed benchmark instance, the causally dominant write carries an old timestamp.

For this reason, the temporal analysis is more useful when ETDL is compared against the temporal demand of the benchmark. We define the required evidence horizon as the age of the oldest annotated gold evidence session for a query. In the 100-question set, this horizon is not always short. The median required horizon is 8.7 days, the mean is 38.0 days, and the maximum is 285.4 days. There are 26 questions whose oldest gold evidence is at least 30 days old, 17 questions at least 90 days old, and 7 questions at least 180 days old.

ETDL is then the realized causal horizon. Table 5 and Figure 2 compare the benchmark-required horizon with the agent’s realized causal horizon. We focus on whether the realized horizon covers the required horizon, because this directly tests whether the causal memory reaches the oldest gold evidence required by the question.

This result changes how we interpret long-range memory. ETDL alone only says that an old memory influenced the answer. The comparison with the required horizon shows whether that influence reaches the temporal part of memory that the benchmark actually requires. Mem0 covers the required

Agent	Timed Dom.	Req. Med. Days	ETDL Med. Days	Covers Required Horizon
Anna	26	8.2	6.8	21/26
SHARE	11	80.2	5.0	4/11
MemoryOS	36	8.5	6.8	29/36
THEANINE	26	8.0	7.7	22/26
Mem0	32	8.7	8.8	32/32

Table 5: Temporal horizon alignment. **Timed Dom.** counts queries with a dominant write and parseable timestamp. **Req. Med. Days** is the median age of the oldest gold evidence for those queries. **Covers Required Horizon** means ETDL reaches the oldest gold evidence horizon within one day.

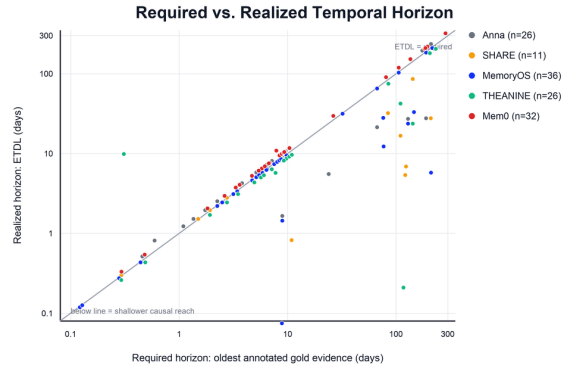


Figure 2: Required evidence horizon versus realized causal horizon. Points on the diagonal mean the dominant memory reaches the oldest annotated gold evidence time; points below the diagonal mean the causal memory is temporally shallower than the oldest gold evidence.

horizon in all timed dominant cases, while MemoryOS, Anna, and THEANINE cover it in about 80–85% of timed cases. SHARE is different: its median required horizon is 80.2 days, but its median ETDL is only 5.0 days, and it covers the required horizon in only 4 of 11 timed cases.

The design implication is that long-term memory should not be managed by recency alone. Some queries require evidence months before the query, so a useful memory system must preserve old but still valid facts. At the same time, the system needs provenance through summarization and compression. Without source-level timestamps, we cannot tell whether the final answer is controlled by the temporally correct evidence or by a newer shortcut.

4.5 Retrieval Is Not Causal Control

Table 6 tests the main causal story. It asks: when the agent finds the right memory, does that memory actually control the final answer? A retrieved-correct dominant case means the gold memory was fetched and removing it had a dominant causal ef-

fect. A retrieved-correct non-dominant case means the agent fetched the gold memory, but that memory did not become the main causal support for the final answer.

This mismatch is the key point of Table 6. MemoryOS, Mem0, and THEANINE all have many retrieved-correct non-dominant cases. This means the right evidence can enter the answering stage, but another memory item, a stale fact, or the model’s generation prior can still control the response. In this way, long-term memory has a RAG-like failure mode: storing the right memory and retrieving the right memory are necessary, but they are not enough. The system also needs to rank, de-noise, and use the retrieved evidence faithfully. This connects to recent RAG work showing that retrieved evidence can conflict with other context or fail to receive proper attribution (Zhang et al., 2025; Abolghasemi et al., 2025).

The design implication is that memory systems need a stronger recall-and-control layer. Aggressively rewriting or summarizing memory can save tokens, but it can also make provenance and conflict resolution harder. A safer design is to preserve source-level evidence when possible, keep temporal context, and make retrieval responsible for ranking the most relevant and current memories. This is also consistent with Mem0’s newer public platform design. The new Mem0 algorithm reports 93.4 accuracy on LongMemEval and moves toward add-only memory writes with multi-signal retrieval rather than overwrite-based updates (Mem0, 2026a,b). We did not evaluate this newer hosted version because our counterfactual replay requires many controlled reruns, while the hosted API was rate-limited and hard to parallelize in our setup. Therefore, our Mem0 result should be read as an evaluation of the older local Mem0 setting, not as an evaluation of the latest hosted Mem0 platform.

Public coding-agent systems show a similar direction, although they are not the same as Mem0. Codex’s open-source memory pipeline extracts per-thread memories, keeps raw memory and rollout-summary artifacts, and then selects and consolidates memories for later use (OpenAI, 2026). Claude Code also exposes memory as editable project, user, local, and auto-memory files, and loads scoped files into context when they are relevant (Anthropic, 2026). These designs support the same lesson from our retrieval-dominance result: long-term agents should preserve auditable memory evidence and improve read-time selection,

Agent	Retrieved Correct		Retrieved Incorrect	
	Dom.	Non-dom.	Dom.	Non-dom.
THEANINE	25	50	0	0
Mem0	33	65	0	0
MemoryOS	33	48	4	15

Table 6: Query-level retrieval-dominance counts for the three strongest memory-dependent systems. **Dom.** means the traced evidence write was causally dominant under counterfactual remove. **Non-dom.** means retrieval happened without dominant causal control.

instead of relying only on opaque conversation summaries or destructive memory rewriting.

5 Conclusion

We use counterfactual memory auditing to trace answer-stage evidence back to original memory writes, remove one traced write, and replay the same question. The results show that downstream accuracy can hide whether the right memory was retrieved and whether it actually controlled the final answer. This means long-term memory systems should be evaluated not only by accuracy, but also by how memories are written, retrieved, and causally used during answering. A good memory system should preserve traceable evidence, retrieve the right information, and make the final answer depend on the right memory rather than stale or competing evidence. Since these requirements depend on the task, memory design should also be application-aligned: factual QA, dialogue, persona, and therapy may need different memory signals.

6 Limitations and Future Work

Evaluation Scale. We evaluate 100 LongMemEval questions across six agents, which is large enough to show consistent patterns but still not the full benchmark. The experiment is expensive because each query requires replaying the history, tracing evidence, and running counterfactual replays. This also limits which systems we can include. For example, we did not evaluate the newest hosted Mem0 platform because the API rate limit made large controlled reruns difficult.

Agent-Specific Instrumentation. The current framework is still agent-specific. For each memory system, we had to write a bridge that records memory writes, carries write identifiers through the agent’s memory pipeline, captures retrieved evidence, and replays the counterfactual state. This

makes the framework useful for analysis, but it is not yet a plug-in tool that can be attached to any agent. A better version would define a general instrumentation API for memory writes, retrieval outputs, lineage, and counterfactual replay.

Counterfactual Action Space. Our counterfactual action space is narrow. In this project, the main action is to block one traced evidence write. This is useful because it gives a clean causal test, but it only covers one kind of memory failure. Future work should support a larger action library. Useful actions include replacing an old memory with an updated value, injecting a conflicting memory, changing the timestamp or retrieval rank of a memory, compressing a raw write into a summary, and separating retrieval-only ablations from generation-only ablations. These actions would test whether the agent can handle memory update, selective forgetting, conflict resolution, recency bias, and summarization loss (Hu et al., 2026; Uddin et al., 2026; Xie et al., 2026).

Prompt and Model Sensitivity. We use a shared answer prompt and GPT-4o-mini for the LLM calls so that the agents are compared under a controlled setting. This makes the comparison cleaner, but the absolute numbers may change under a different prompt, model, or evaluator. Future work should test whether the same dominance and retrieval-mismatch patterns hold across multiple prompts and model backends.

Replay Benchmark Realism. LongMemEval is a replay benchmark rather than a real months-long deployment. This matters most for the temporal analysis. A large ETDL means that the dominant write has an old timestamp inside the replayed history, not that the deployed agent actually stored and used that memory for months in a live setting. Future work should test the same counterfactual auditing framework in real longitudinal deployments or interactive memory benchmarks.

References

- Amin Abolghasemi, Leif Azzopardi, Seyyed Hadi Hashemi, Maarten de Rijke, and Suzan Verberne. 2025. [Evaluation of attribution bias in generator-aware retrieval-augmented large language models](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 21105–21124, Vienna, Austria. Association for Computational Linguistics.
- Anthropic. 2026. How Claude remembers your project. <https://code.claude.com/docs/en/memory>. Accessed May 8, 2026.
- Jiayang Cheng, Dongyu Ru, Lin Qiu, Yiyang Li, Xuezhi Cao, Yangqiu Song, and Xunliang Cai. 2026. [AMemGym: Interactive memory benchmarking for assistants in long-horizon conversations](#). In *Proceedings of the Fourteenth International Conference on Learning Representations*.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. [Mem0: Building production-ready AI agents with scalable long-term memory](#). *arXiv preprint arXiv:2504.19413*.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. 2026. [Evaluating memory in LLM agents via incremental multi-turn interactions](#). In *Proceedings of the Fourteenth International Conference on Learning Representations*.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. [Memory OS of AI agent](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25961–25970, Suzhou, China. Association for Computational Linguistics.
- Eunwon Kim, Chanho Park, and Buru Chang. 2025. [SHARE: Shared memory-aware open-domain long-term dialogue dataset constructed from movie script](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14474–14498, Vienna, Austria. Association for Computational Linguistics.
- Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. 2025. [Hello again! LLM-powered personalized agent for long-term dialogue](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5259–5276, Albuquerque, New Mexico. Association for Computational Linguistics.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. [Evaluating very long-term conversational memory of LLM agents](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.
- Mem0. 2026a. Benchmarking Mem0’s token-efficient memory algorithm. <https://mem0.ai/research>. Accessed May 8, 2026.
- Mem0. 2026b. Platform: Migrating to the new memory algorithm. <https://docs.mem0.ai/migration/platform-v2-to-v3>. Accessed May 8, 2026.
- Kai Tzu-iunn Ong, Namyoung Kim, Minju Gwak, Hyungjoo Chae, Taeyoon Kwon, Yohan Jo, Seungwon Hwang, Dongha Lee, and Jinyoung Yeo. 2025. [Towards lifelong dialogue agents via timeline-based memory management](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8631–8661, Albuquerque, New Mexico. Association for Computational Linguistics.
- OpenAI. 2024. Prompt caching in the API. <https://openai.com/index/api-prompt-caching/>. Accessed May 8, 2026.
- OpenAI. 2026. Codex memories pipeline. <https://github.com/openai/codex/blob/main/code-x-rs/core/src/memories/README.md>. Accessed May 8, 2026.
- Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. 2026. [From recall to forgetting: Benchmarking long-term memory for personalized agents](#). *arXiv preprint arXiv:2604.20006*.
- Ming Wang, Peidong Wang, Lin Wu, Xiaocui Yang, Daling Wang, Shi Feng, Yuxin Chen, Bixuan Wang, and Yifei Zhang. 2025. [AnnaAgent: Dynamic evolution agent system with multi-session memory for realistic seeker simulation](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23221–23235, Vienna, Austria. Association for Computational Linguistics.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. [LongMemEval: Benchmarking chat assistants on long-term interactive memory](#). In *Proceedings of the Thirteenth International Conference on Learning Representations*.
- Weiwei Xie, Shaoxiong Guo, Fan Zhang, Tian Xia, Xue Yang, Lizhuang Ma, Junchi Yan, and Qibing Ren. 2026. [MemEvoBench: Benchmarking memory misevolution in LLM agents](#). *arXiv preprint arXiv:2604.15774*.
- Qinggang Zhang, Zhishang Xiang, Yilin Xiao, Le Wang, Junhui Li, Xinrun Wang, and Jinsong Su. 2025. [FaithfulRAG: Fact-level conflict modeling for context-faithful retrieval-augmented generation](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21863–21882, Vienna, Austria. Association for Computational Linguistics.

A Unified Answer Prompt

LongMemEval, SHARE, THEANINE, and LD-Agent place full prompts in the appendix as prompt figures or prompt tables. Following this format, we include the shared answer prompt used by our bridges.

System prompt
You are answering a long-term memory benchmark question.
Use only the provided evidence.
Do not role-play. Do not continue the conversation. Do not use outside knowledge.
Rules:
1. Base the answer only on the provided evidence.
2. If the evidence includes timestamps, dates, or temporal order, use them explicitly.
3. If multiple evidence items conflict, prefer the evidence that is most directly relevant and temporally valid for the question.
4. If the evidence is insufficient, answer exactly: I don't know.
5. Return only the final concise answer, with no explanation.
User prompt template
Question:
{query_with_date}
Retrieved Evidence:
{evidence_block}
Return only the final concise answer.
Evidence block template
[Memory {idx} source={source} date={timestamp}]
{memory_text}
If no evidence is available, the evidence block is:
[Memory 1 source=none date=unknown]
(none)

Figure 3: Unified answer prompt used across the agent bridges.

B Estimated API Cost

Table 7 shows the input-side API cost of the 100-question run. We estimate this from the prompt-cache log lines produced by the OpenAI wrapper. These logs record prompt_tokens and cached_tokens for successful gpt-4o-mini calls. We use OpenAI’s listed prices for gpt-4o-mini: \$0.15 per 1M input tokens and \$0.075 per 1M cached input tokens (OpenAI, 2024). The table does not include output-token cost, because this run did not consistently log completion_tokens for every agent. Therefore, the dollar values should be read as lower-bound input-cost estimates rather than exact billing numbers.

Agent	API Calls	Input Tokens (M)	Cached Tokens (M)	Input Cost	Cost / Question
AnnaAgent	1,271	20.0	8.7	\$2.34	\$0.02
SHARE	97,953	198.7	50.7	\$26.00	\$0.26
MemoryOS	949,595	838.5	3.9	\$125.48	\$1.25
LD-Agent	268,634	144.5	0.0	\$21.68	\$0.22
THEANINE	852,119	4,237.3	2,402.6	\$455.40	\$4.55
Mem0	98,561	885.4	694.7	\$80.71	\$0.81
Total	2,268,133	6,324.4	3,160.7	\$711.61	\$7.12

Table 7: Lower-bound input-side API cost estimate for the 100-question counterfactual run. Output-token cost is not included because completion tokens were not consistently logged in this run.

The cost comes from different parts of each memory pipeline. AnnaAgent is cheap in this experiment because the bridge mostly uses compact memory views and a small number of LLM calls for retrieval

and final answering. SHARE spends more because it repeatedly extracts, updates, and selects shared dialogue memories. MemoryOS spends heavily during memory writing: short-term memory eviction triggers multi-topic summaries, keyword extraction, and mid-term memory updates. LD-Agent uses LLM calls for session/context summaries and final answering, but its memory operations are less dense than MemoryOS and THEANINE. THEANINE is the most expensive because its timeline-based design repeatedly summarizes sessions, constructs linked memories, and refines retrieved timeline paths before response generation. Mem0 is also non-trivial because memory insertion performs inference over many message pairs, although prompt caching reduces part of the repeated input cost.

This cost view is not a performance metric, but it shows an operational tradeoff. In real applications, memory writing can often happen asynchronously, so latency may be hidden from the user. However, asynchronous execution does not remove API cost. A memory design with many summarization, refinement, or update calls can still be expensive even if it gives better accuracy. This suggests that practical memory systems should optimize not only accuracy and robustness, but also how many LLM calls are needed to maintain the memory state.